SKETCHANIM: REAL-TIME SKETCH ANIMATION TRANSFER FROM VIDEOS

Anonymous ICME submission

ABSTRACT

Animation of hand-drawn sketches is an adorable art. It allows the animator to generate animations with expressive freedom and requires significant expertise. In this work, we introduce a novel sketch animation framework designed to address inherent challenges, such as motion extraction, motion transfer, and occlusion. The framework operates on a video input featuring a moving object, utilizing a robust motion transfer technique to animate the corresponding sketch. Comparative evaluations demonstrate the superior performance of our method over existing sketch animation techniques. Notably, our approach exhibits a higher level of user accessibility in contrast to conventional sketch-based animation systems, positioning it as a promising contributor to the field of sketch animation.

***Index Terms*—** Sketch Animation, Point Tracking, Motion Transfer, Linear Blend Skinning.

1. INTRODUCTION

Sketches serve as a great medium for visual communication and animating sketches can add dynamism and interactivity. Traditional tools for animating sketches demand a unique set of skills to bring hand-drawn characters to life. When attempting to animate a character, it is common for an artist to refer to some existing videos and try to replicate a similar video motion to the character. However, this process is often time-consuming, and an inaccurate replication of motion may make the character's movement look non-fluent or unnatural, especially for those with little to no experience in animation.

Character animation has been an area of extensive research in recent years. Previous work includes that of Xing et al. [1] who proposed a method that autocompletes a set of continuously repeated strokes during sketching and performs a keyframe-based animation. After the user draws the sketch outlines for each frame and adds details to the initial frame, those details are replicated across all frames. TraceMove [2] provides a data-assistant user interface that enables novice animators in sketching and 2D character animation. The method involves the user providing skeleton points as input, and then utilizes a motion capture dataset to predict the skeleton in the following frames. Live-Sketch [3] extracts dynamic deformation from videos as moving control points and transfers the motion to sketches while addressing certain challenges such as extracting motion from videos, video-to-sketch alignment, and motion transfer.

Recent advancements in sketch animation utilize neural network-based techniques like CharacterGAN [4], that relies on keypoints added manually by the user. Another approach, AnimationDrawing [5], uses video motion and applies it to the sketch. However, it has limitations, supporting only the animation of biped characters and generating artifacts in instances of inaccurate joint estimation and occlusion.

In this work, we propose a method for animating sketches by applying the motion from a driving video to the sketch. Our algorithm takes as input a sketch to be animated, a driving video from which the motion is to be applied, and a skeleton of the character from either the sketch or the first frame of the video. It then generates an animated sketch with the motion of the video applied to it. Our method offers multiple advantages at certain stages. It supports the animation of characters with general skeleton structures, not restricted to having biped or quadruped skeleton structures. Our motion extraction method, combined with the support of generalized skeletons, allows the user to apply animation to inanimate object sketches. Our approach generates animations with minimal loss compared to many other recent techniques. The main contributions of our work are as follows

- We provide SketchAnim, an automatic, user-friendly, and efficient sketch animation framework capable of handling different types of motions such as biped, quadruped and inanimate, and
- A novel motion extraction and motion transfer approach for sketch animation that handles distortion and unnatural artifacts despite being different shapes of sketch and video.

2. RELATED WORK

Significant work has been done in sketch animation in recent years. The major difficulties in handling geometric deformations, partial occlusions, in-plane and out-of-plane rotations, and complex backgrounds make this task highly challenging. Recent improvements with the sketch animation attempt to create interactive animations by adding dynamic motion. The approach of Bregler et al. [6] is a keyframe-based technique that tracks the cartoon motion and maps to the sketch or line

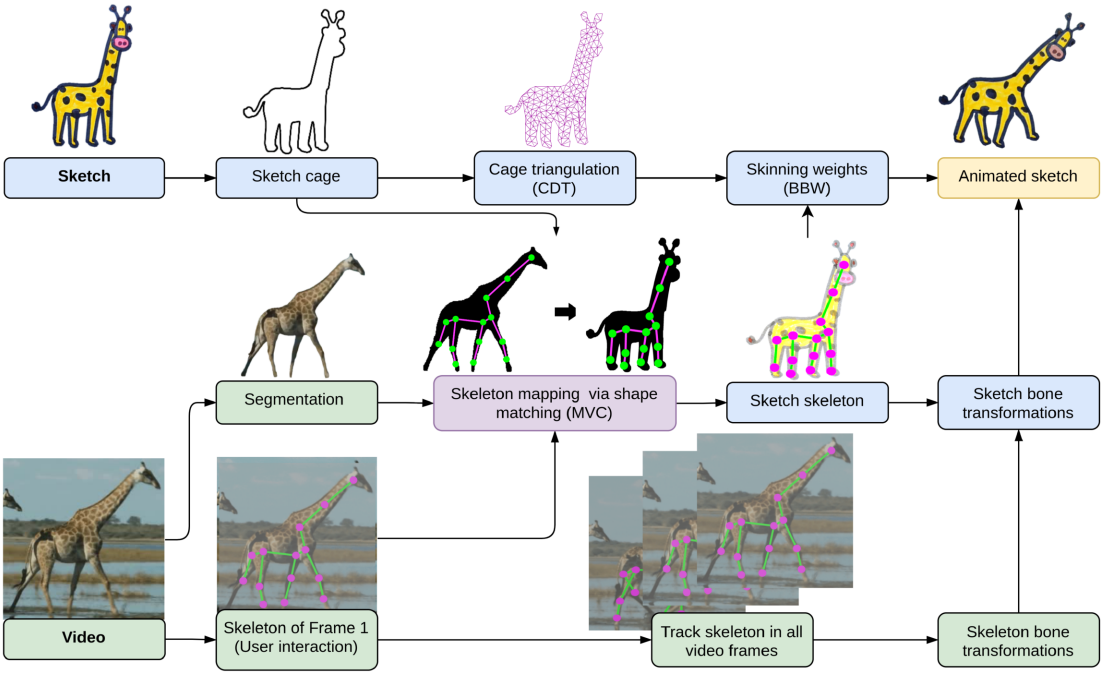

Fig. 1: Overview of our SketchAnim framework for sketch based animation.

drawing. Few sketch-based animation methods [2, 1] not only animate the static object but also assist in sketching, however they require manual user input at multiple stages and are purely data-driven. Santosa et al. [7] used optical flow for the animation, but it requires a similar structure of input sketch as in the exemplar video. Several methods [8, 3] for sketch animation use motion from a reference video. However, these methods cannot handle complex video motion as they fail to map the accurate motion on the sketch. Further, these methods require manual user input at multiple stages. Ben-Zvi et al. [9] proposed a data-driven method that instead of extracting the video's motion trajectories and transferring them to the input sketch performs it as style transfer technique. Live-Sketch [3] extracts the sparse motion trajectories from the video and transfers them to the sketch using control point tracking. However, this method also needs manual user input and cannot handle the motion for smooth regions. The hand-drawn character animation method [5] performs hand-drawn character animation using video motion, but suffers from motion estimation in smooth regions and only handles biped motion.

The two primary categories of deep learning techniques for motion transfer are model-based and model-free approaches. Model-based approaches primarily focus on pose-guided human image generation [10, 11]. The other model-based methods [12, 13] use facial landmark detector to extract the pose of the driving image as guidance information. These methods require prior supervision of the moving object, such as facial landmarks and human pose. On the other hand, model-free approaches do not require any supervision or labeled data. Since the video generation problem is close to the future frame prediction problem, X2Face [13] propose a deep learning method that uses a dense motion field to deform the input face and generate the deformed image output. However, this is a data-driven method and is limited to faces only. In the past years, many deep learning methods [14, 15] have been proposed that are unrestricted to a specific object and do not require labeled data. MonkeyNet [14] is a self-supervised method that uses a learned unsupervised keypoint-based dense motion field using which it transfers the motion pattern of the driving video to the image. This method fails when the driving video poses drastic pose changes. FOMM [15] adds local affine transformation over the learned keypoints to generate the animated image which improves the motion transfer quality. However, it fails for the sketch modality since it is a keypoint transformation-based motion model only suitable for image animation. Our proposed method, SketchAnim, is user-friendly and needs only sketch input and a reference video with a skeleton on the first frame of the video. It does not require additional manual user inputs, and handles the topology structure even though the object structure is different in the video from the sketch.

3. APPROACH

We propose *SketchAnim*, an easy to use and robust framework for sketch animation using motion from a driving video. The algorithm begins by obtaining user-provided input, including

a sketch $\mathcal{S}$, a driving video $\mathcal{V}$ containing an object, and a user-provided skeleton $\mathcal{K}_v$ on the first frame of the driving video. The driving video may be another animated or a real-world video that provides a realistic character motion for the sketch. The video skeleton $\mathcal{K}_v$ is tracked across all frames of the video. Subsequently, bone-joint angles are estimated for the motion skeleton. The video skeleton is mapped to the sketch to compute a sketch skeleton $\mathcal{K}_s$ and skinning weights are computed using bounded biharmonic weights[16]. The algorithm then calculates the bone transformations required for sketch animation through linear blend skinning. Finally, the transformations are applied to the sketch to generate an animated output video $\mathcal{V}_s$, effectively portraying the given sketch in accordance with the dynamics of the driving video sequence. The output video is presented to the user as the animated result, encapsulating the essence of the original sketch within the context of the driving video. Algorithm 1 outlines the primary steps of our approach and Fig. 1 shows the overall framework for sketch animation.

Algorithm 1: Video-to-Sketch motion transfer

Input: Sketch $\mathcal{S}$, driving video $\mathcal{V}$, and video skeleton $\mathcal{K}_v$
Output: Sketch video $\mathcal{V}_s$
Segment sketch boundary polygon $\mathcal{B}_s$ using SAM
Compute triangulation $\mathcal{T}_s$ (CDT) of $\mathcal{B}_s$
Compute sketch skeleton $\mathcal{K}_s$ from $\mathcal{K}_v$ using MVC
Segment the first frame of the video $\mathcal{B}_v$ using SAM
foreach frame *in* $\mathcal{V}$ **do**
  Track each vertex in $\mathcal{K}_v$ to the next frame
  Estimate bone transformations for $\mathcal{K}_v$
  Map bone transformations from $\mathcal{K}_v$ to $\mathcal{K}_s$
Compute BBW weights for $\mathcal{T}_s$ and $\mathcal{K}_s$
Deform $\mathcal{S}$ using LBS to produce $\mathcal{V}_s$

3.1. Skeleton mapping and motion extraction

Our sketch animation approach takes as input a sketch $\mathcal{S}$ image and a driving video $\mathcal{V}$ containing n frames. We use the Segment Anything Model (SAM) [17] to perform segmentation of the characters from the sketch and the first video frame. We resample the boundaries of these segmentation regions with points spaced d distance apart as polygons $\mathcal{B}_s$ and $\mathcal{B}_v$ respectively for the sketch image and the video frame.

The skeleton $\mathcal{K}_v$ of the first frame of the video is taken as an input from the user. We map the vertices of this skeleton to the sketch skeleton $\mathcal{K}_v$ while keeping the topologies of the two identical. Our skeleton mapping approach is based on shape matching between boundaries $\mathcal{B}_v$ and $\mathcal{B}_s$. To start with, $\mathcal{B}_s$ and $\mathcal{B}_v$ do not have much correlation as they were constructed independently of each other. We use the neural deformation pyramid approach of Li and Harada [18] to deform the video boundary $\mathcal{B}_v$ and get a close approximation of $\mathcal{B}_s$. This results in a new sketch boundary $\tilde{\mathcal{B}}_s$ whose points are in correspondence with $\mathcal{B}_v$ and $|\tilde{\mathcal{B}}_s| = |\mathcal{B}_v|$. Once the shape correspondence between the two boundaries is established, we transform the vertices of the video skeleton $\mathcal{K}_v$ to those of $\mathcal{K}_s$ via mean value coordinates (MVC) [19]. To perform the same, we calculate MVC $\alpha_v \in \mathbb{R}^{|\mathcal{B}_v|}$ of a vertex $q_v \in \mathcal{K}_v$ with respect to the boundary polygon $\mathcal{B}_v$ and use $\tilde{\mathcal{B}}_s$ to calculate the corresponding skeleton point q_s in the sketch skeleton $\mathcal{K}_s$ as

$$q_s = \sum_{j=1}^{|\tilde{\mathcal{B}}_s|} p_s^{(j)}\, \alpha_v^{(j)}, \forall p_s^{(j)} \in \tilde{\mathcal{B}}_s.$$

In order to transfer the video motion to the sketch, we also need to capture the motion of $\mathcal{K}_v$ across frames of the video. This can be achieved with point tracking in the video, however the primary challenge here comes from occlusions and error propagation across frames. We require a sparse tracking method that is reasonably resilient to occlusions and propagated errors. We employ the CoTracker [20] neural network model to track vertices of $\mathcal{K}_v$ across frames keeping the skeleton topology fixed.

3.2. Motion Transfer

We would like to derive a sequence of sketch skeletons $\{\mathcal{K}_s^{(i)} \mid i = 1, \ldots, n\}$ from the sequence video skeletons $\{\mathcal{K}_v^{(i)}\}$ computed previously. While there are many ways to do so, for best results one should ensure that the skeleton motion of the video is captured by the sketch accurately and that the sketch proportions do not get deformed unnaturally.

For the k^{th} frame of the video sequence, consider the video skeleton $\mathcal{K}_v^{(k)}$ and as a tree with the root vertex chosen as the vertex with highest degree. Since the video and sketch skeleton trees are isomorphic, we choose the same vertex to be the root node in both across all the frames. Our motion mapping begins with setting the position of the root vertex in the sketch skeleton to be that of the root vertex of the video skeleton. Thereafter we proceed in a breadth first search (BFS) fashion to compute orientation and length of each edge in the sketch skeleton from the corresponding edge in the video skeleton. We ensure that the angle of each edge in the sketch skeleton is the same as that of the video skeleton edge with respect to the coordinate axes. If there is scaling in the video frame edge then the same scaling factor is applied to the sketch edge.

More formally, let an edge in the video skeleton $\mathcal{K}_v^{(k)}$ be $e_v^{(k)} = (a_v^{(k)}, b_v^{(k)})$, the corresponding edge in the sketch skeleton $\mathcal{K}_s^{(k)}$ be $e_s^{(k)} = (a_s^{(k)}, b_s^{(k)})$. The root nodes be $r_v^{(k)}$ and $r_s^{(k)}$ for video and the sketch skeletons respectively, and we set $r_s^{(k)} = r_v^{(k)}$. At any given time, the vertex position $a_s^{(k)}$ comes from the previous computation in the BFS pass,

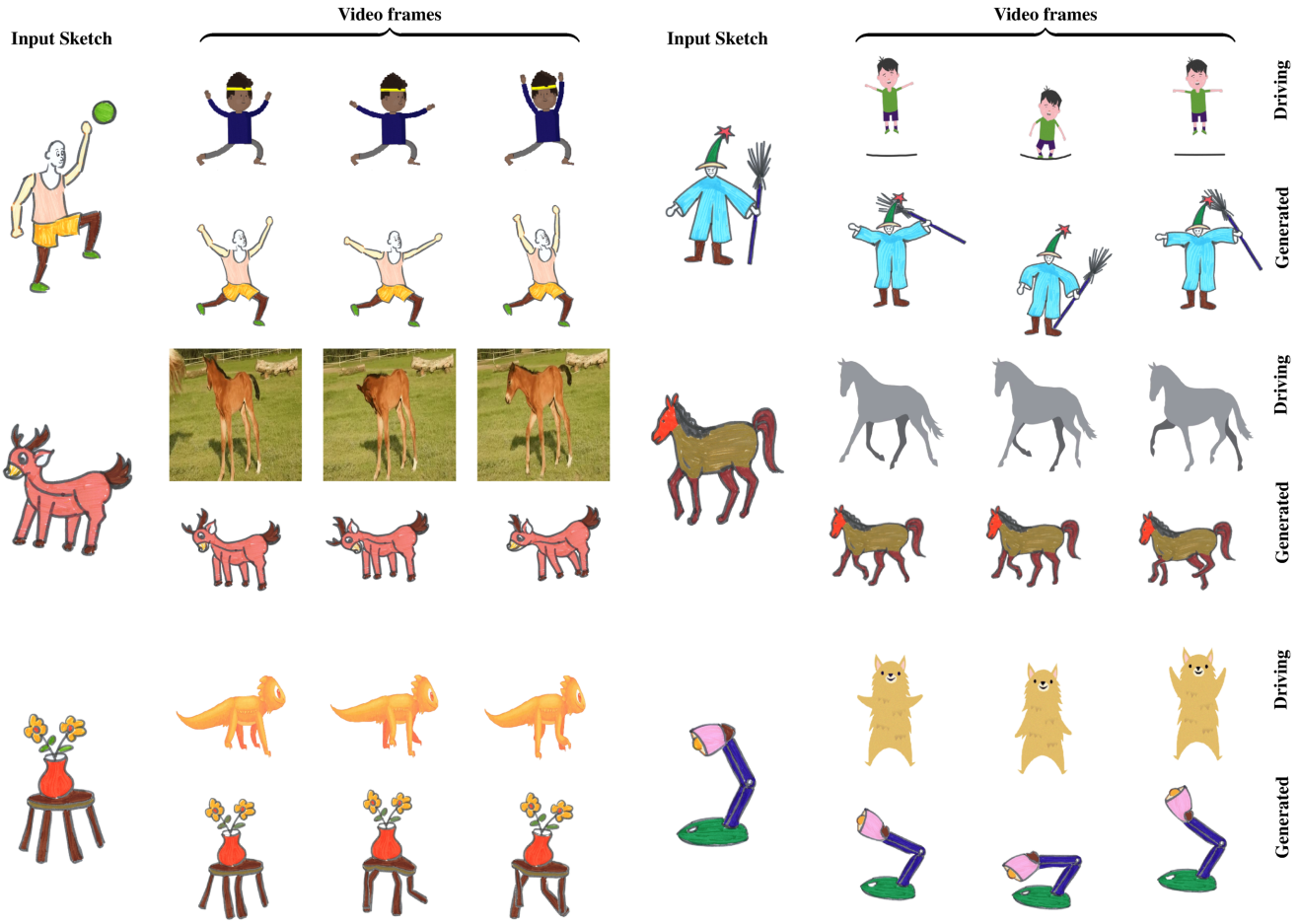

Fig. 2: SketchAnim results on biped, quadruped and non-living sketches.

and the position of vertex $b_s^{(k)}$ is computed as

$$b_s^{(k)} = a_s^{(k)} + \frac{\|b_s^{(1)} - a_s^{(1)}\|_2}{\|b_v^{(1)} - a_v^{(1)}\|_2}(b_v^{(k)} - a_v^{(k)}).$$

We perform this motion mapping for each frame of video skeleton to obtain a sequence of sketch skeletons.

3.2.1. Resolving self-occlusions

Self occlusions in 2D animation are difficult to handle if depth ordering of different part of the object are not known. We resolve occlusions by creating depth order for the skeleton vertices first and then propagating these to the mesh during skinning. To resolve occlusions, we add a discrete depth value to each vertex in the skeleton starting with a zero value at the root vertex. Every other vertex is given a value equal to the number of edges it is away from the root. We assign a negative sign to all the vertices for which the corresponding vertices in the video skeleton were found to be occluded in one or more frames during tracking. This gives us correct depth order of skeleton vertices in the sketch. During skinning animation, we transfer the depth values from skeleton vertices to mesh vertices based on the skinning weights in a fashion similar to linear blend skinning, and use these depth values as the z-coordinate of mesh vertices in rendering.

After the video motion is transferred to sketch skeleton, we animate the sketch. To do so, we first triangulate the sketch boundary polygon and compute a constrained Delaunay triangulation (CDT) of the given boundary $\mathcal{B}_s$. Additional points are added to the triangulation by Poisson disk sampling the interior of the boundary. Smooth skinning weights for this triangulation and the skeleton $\mathcal{K}_s^{(1)}$ are computed using the Bounded Biharmonic Weights (BBW) [16] method. The triangulation is then deformed using linear blend skinning (LBS) and texture mapped with the sketch image to generate each frame of the animated sketch.

4. RESULTS AND ANALYSIS

We test SketchAnim on a computer with an Intel i5 CPU, 16 GB RAM, and Nvidia GeForce GTX 1080 graphics card. We

created colorized sketches to illustrate sketch animation capabilities of SketchAnim. These included about 50 hand-drawn sketch samples with different categories - biped, quadruped and non-living. We collected samples of driving videos with the Creative Commons license available from the internet. The selected cartoon and natural videos contained a central character/object completely visible in all the frames. Some quadruped samples were also selected from the MGif [15] dataset. The SketchAnim framework is implemented in Python as an interactive application. The source code for our implementation will be made available for research purposes.

Fig. 2 shows animated sequences for three different categories of sketches: biped, quadruped, and non-living. In the biped example sketches, the hand movements correspond to the motion of the video object as depicted in the videos. The quadrupedal motion of a horse is evident in the depicted sketches, showcasing the running movement. In non-living sketches, the leg motion of a lizard is synchronized with the table poles, mirroring the lizard's walking style. In another scenario, the lamp exhibits jumping and bending motions in accordance with the hand movements of a bear.

Our method is computationally very efficient and more accurate compared to other state-of-the-art methods. The method of Siarohin et al. [15] uses keypoint transformation for animation that fails in cross-domain motion transfer and generates artifacts during sketch animation. The approach of Smith et al. [5] is close to our method and generates promising results but it is limited only to the biped motion category whereas our method seamlessly handles other motion categories very well. Our method is more robust and accurate regarding motion transfer, reduced distortion, and precise motion estimation by handling occlusion cases. It also handles the artifacts of shape distortion arising due to non-linear complex motions. For quantitative comparison, we employ the Frechet Inception Distance (FID) [21] as the evaluation metric. FID is utilized to gauge the overall quality of the generated frames with respect to the original sketch. It involves comparing the features of the generated frames with those of real images and calculating the distance between them. Table 1 shows comparison with the above two methods and shows that our method outperforms these. Figs. 3 and 4 show visual comparison with these methods. It is clear that the subtle motion of body parts in the characters are very well captured by our method whereas the other two fail to model these. The cases of occlusions are also well handled with our method.

Table 1: FID score (lower the better) for comparison with other methods.

Method	Biped	Quadruped	Inanimate
Smith et al. [5]	201.65	-	-
Siarohin et al. [15]	186.37	160.01	190.37
Ours	**150.25**	**140.00**	**176.89**

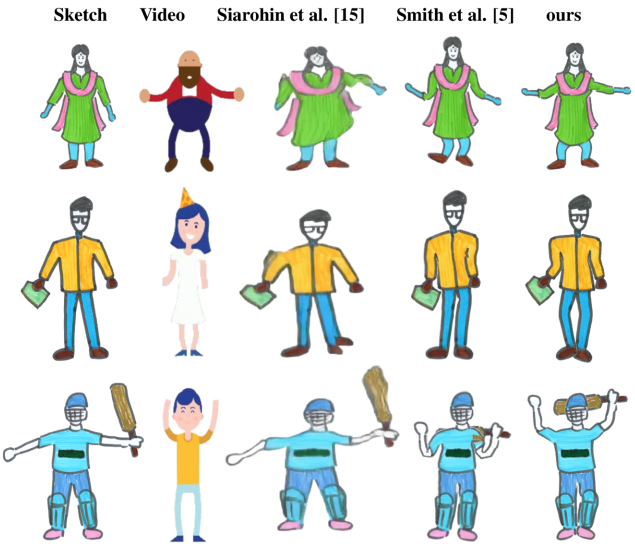

Fig. 3: Comparison of biped samples with state-of-the-art methods Siarohin et al. [15] and Smith et al. [5].

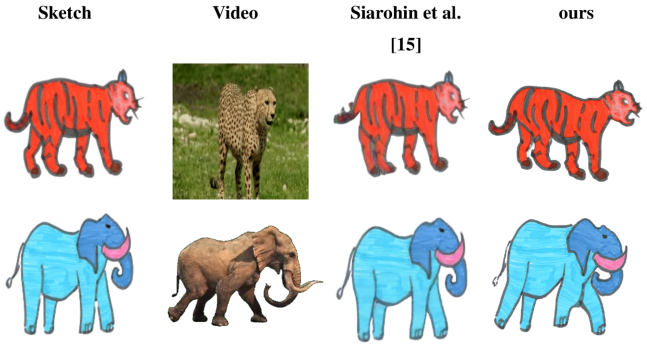

Fig. 4: Comparison of quadruped animation.

5. CONCLUSION

In this paper, we introduced a method for animating sketches based on a static sketch and a reference video. The animation process involves applying the motion from the reference video to animate the sketch. Users have the flexibility to choose a specific animation for the sketch, providing a direct reference video input. Furthermore, we take minimal user input in the form of drawing a skeleton on a video frame. This approach enables the animation of sketches with arbitrary shapes, removing restrictions to specific categories such as biped or quadruped.Our method successfully captures subtle motion from the given video and performs better than similar methods in terms of quality of animation. While our proposed framework exhibits notable strengths, it does have certain limitations. For instance, it may encounter chal-

lenges in handling motion when there is no direct correspondence between the video object and the sketch. Tracking failures can arise in complex motion scenarios within the video, and instances of occlusion. In such cases, errors may propagate in skeleton mapping and motion transfer. Although our method effectively manages self-occlusion cases, it does not autonomously complete missing sketch details in specific scenarios when there is overlap between parts of the body. We wish to handle more complex scenarios by building upon this work.

References

[1] Jun Xing, Li-Yi Wei, Takaaki Shiratori, and Koji Yatani, "Autocomplete hand-drawn animations," *ACM Transactions on Graphics (TOG)*, vol. 34, no. 6, pp. 1–11, 2015.

[2] Priyanka Patel, Heena Gupta, and Parag Chaudhuri, "Tracemove: A data-assisted interface for sketching 2d character animation," in *Proceedings of the 11th Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications: Volume 1: GRAPP*, 2016, pp. 191–199.

[3] Qingkun Su, Xue Bai, Hongbo Fu, Chiew-Lan Tai, and Jue Wang, "Live sketch: Video-driven dynamic deformation of static drawings," in *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, 2018, pp. 1–12.

[4] Tobias Hinz, Matthew Fisher, Oliver Wang, Eli Shechtman, and Stefan Wermter, "Charactergan: Few-shot keypoint character animation and reposing," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2022, pp. 1988–1997.

[5] Harrison Jesse Smith, Qingyuan Zheng, Yifei Li, Somya Jain, and Jessica K Hodgins, "A method for animating children's drawings of the human figure," *ACM Transactions on Graphics*, vol. 42, no. 3, pp. 1–15, 2023.

[6] Christoph Bregler, Lorie Loeb, Erika Chuang, and Hrishi Deshpande, "Turning to the masters: Motion capturing cartoons," *ACM Transactions on Graphics (TOG)*, vol. 21, no. 3, pp. 399–407, 2002.

[7] Stephanie Santosa, Fanny Chevalier, Ravin Balakrishnan, and Karan Singh, "Direct space-time trajectory control for visual media editing," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 2013, pp. 1149–1158.

[8] Robert Held, Ankit Gupta, Brian Curless, and Maneesh Agrawala, "3d puppetry: a kinect-based interface for 3d animation.," in *UIST*. Citeseer, 2012, vol. 12, pp. 423–434.

[9] Nir Ben-Zvi, Jose Bento, Moshe Mahler, Jessica Hodgins, and Ariel Shamir, "Line-drawing video stylization," in *Computer Graphics Forum*. Wiley Online Library, 2016, vol. 35, pp. 18–32.

[10] Caroline Chan, Shiry Ginosar, Tinghui Zhou, and Alexei A Efros, "Everybody dance now," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 5933–5942.

[11] Zhenglin Geng, Chen Cao, and Sergey Tulyakov, "3d guided fine-grained face manipulation," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 9821–9830.

[12] Sungjoo Ha, Martin Kersner, Beomsu Kim, Seokjun Seo, and Dongyoung Kim, "Marionette: Few-shot face reenactment preserving identity of unseen targets," in *Proceedings of the AAAI conference on artificial intelligence*, 2020, vol. 34, pp. 10893–10900.

[13] Olivia Wiles, A Koepke, and Andrew Zisserman, "X2face: A network for controlling face generation using images, audio, and pose codes," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 670–686.

[14] Aliaksandr Siarohin, Stéphane Lathuilière, Sergey Tulyakov, Elisa Ricci, and Nicu Sebe, "Animating arbitrary objects via deep motion transfer," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 2377–2386.

[15] Aliaksandr Siarohin, Stéphane Lathuilière, Sergey Tulyakov, Elisa Ricci, and Nicu Sebe, "First order motion model for image animation," *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[16] Alec Jacobson, Ilya Baran, Jovan Popovic, and Olga Sorkine, "Bounded biharmonic weights for real-time deformation.," *ACM Trans. Graph.*, vol. 30, no. 4, pp. 78, 2011.

[17] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al., "Segment anything," *arXiv preprint arXiv:2304.02643*, 2023.

[18] Yang Li and Tatsuya Harada, "Non-rigid point cloud registration with neural deformation pyramid," *Advances in Neural Information Processing Systems*, vol. 35, pp. 27757–27768, 2022.

[19] Michael S Floater, "Mean value coordinates," *Computer aided geometric design*, vol. 20, no. 1, pp. 19–27, 2003.

[20] Nikita Karaev, Ignacio Rocco, Benjamin Graham, Natalia Neverova, Andrea Vedaldi, and Christian Rupprecht, "Cotracker: It is better to track together," *arXiv preprint arXiv:2307.07635*, 2023.

[21] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter, "Gans trained by a two time-scale update rule converge to a local nash equilibrium," *Advances in neural information processing systems*, vol. 30, 2017.